

Lost&Found; AI

An intelligent lost-property website that connects lost and found reports using semantic similarity, location, time and category data.

The goal

Make reporting an item effortless, then automatically surface the most likely matching reports. The product should feel useful in the first 30 seconds of a demo.

Why this works as a hackathon project

- Clear problem: people lose things, while found-item posts are fragmented and difficult to search.
- Clear workflow: report an item, store it, compare it, show likely matches.
- Meaningful MongoDB role: documents store reports; Vector Search powers semantic matching; indexes and aggregation support filters and scoring.
- Strong demo: two slightly different descriptions can still be matched, showing something keyword search would miss.

MVP	Demo	Future
Report lost/found items and browse them.	Create two similar reports and reveal a high-confidence match.	Image matching, maps, notifications and verified handover.

1. Product workflow

The MVP should focus on one complete user journey rather than many half-finished features.

1	Report User selects Lost or Found, then enters title, description, category, location, date/time and optional image URL.
2	Store The report is saved as a MongoDB document with consistent fields and timestamps.
3	Match For a lost item, search only found reports; for a found item, search only lost reports. Semantic similarity becomes the main signal.
4	Rank Combine semantic similarity with category, location and time proximity to generate a transparent match score.
5	Reunite Show the strongest potential matches and let the user inspect the report details.

Core pages

Home /

Hero: "Lost something? Let AI help find it."

Two primary actions: I Lost Something and I Found Something.

Show recent reports below.

Report /report

One form for both lost and found items. Make the item type visually obvious. On submit, save to MongoDB and redirect to the new item page.

Browse /items

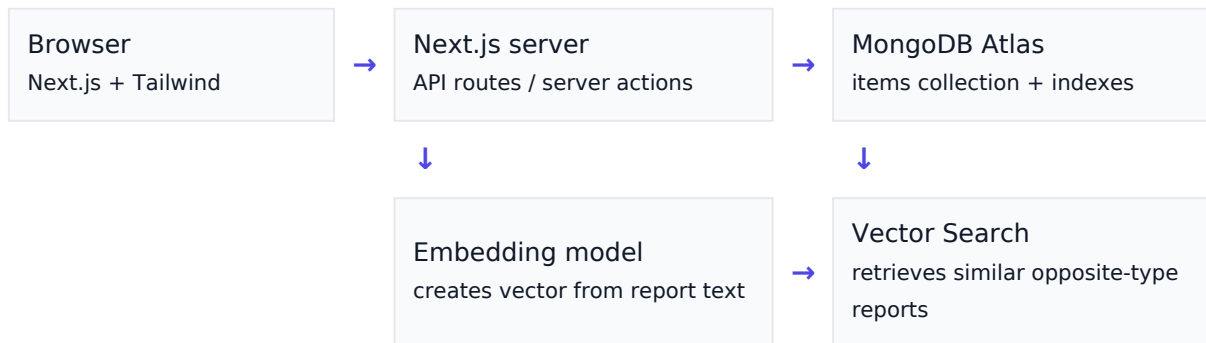
Search, filter by Lost/Found and category, sort newest first, and display consistent report cards.

Item /items/[id]

Full report details plus a dedicated Potential Matches area. This becomes the main AI demo screen.

2. Technical architecture

Keep the stack small enough to finish, while making MongoDB central to the product.



Suggested stack

Layer	Choice	Why
Frontend	Next.js + TypeScript + Tailwind	Fast to build, responsive UI, server and client in one codebase.
Database	MongoDB Atlas	Flexible reports, search, indexing and vector retrieval in one platform.
Database driver	Official mongodb npm package	Minimal dependency surface and direct MongoDB access.
Matching	MongoDB Vector Search	Find semantically similar descriptions, not only exact keywords.
Icons	Lucide	Fast, consistent visual polish.

MongoDB features to highlight to judges

- Document model: each report naturally contains nested location data, timestamps, status and optional AI fields.
- Vector Search: retrieve conceptually similar reports even when users use different wording.
- Indexes: support fast filtering by type, category, status and time.
- Aggregation: combine signals or summarize report activity.
- Optional later: geospatial indexes, Change Streams and TTL indexes for stale temporary data.

3. Data model and matching logic

A single items collection is enough for the MVP. Lost and found reports share the same shape and are distinguished by the type field.

```
{
  _id: ObjectId,
  type: "lost" | "found",
  title: "Black AirPods Pro Case",
  description: "Lost near the library entrance",
  category: "Electronics",
  location: { name: "DCU Library" },
  eventDate: ISODate(...),
  imageUrl: "...",           // optional
  status: "open",
  embedding: [ ... ],       // phase 2
  createdAt: ISODate(...),
  updatedAt: ISODate(...)
}
```

Match score

Keep the first scoring model understandable. The goal is not scientific perfection; it is a transparent ranking the team can explain.

Signal	Weight	Example
Semantic description similarity	60%	"Black AirPods case" can match "dark wireless earbud charging case".
Location proximity / similarity	20%	Library entrance and library lobby are close.
Time proximity	10%	Found 25 minutes after the loss is more plausible than 3 weeks later.
Same category	10%	Electronics ↔ Electronics strengthens the match.

POTENTIAL MATCH

93%

Lost: Black AirPods Pro Case
DCU Library · 12:35

Found: Dark wireless earbud case
Library entrance · 13:02

Description 96% Location 98% Time 84% Category 100%

4. Build plan and 3-minute demo

The project should be built in phases. A finished CRUD website is more valuable than an ambitious matching engine that never becomes demoable.

Phase	Deliverable	Done when...
1. Foundation	Next.js app, layout, MongoDB connection	A report can be saved and read back from Atlas.
2. Core workflow	Report form, browse page, item detail page	Lost and found items can be created and viewed end-to-end.
3. Matching	Embeddings + Vector Search + ranking	An item page shows sensible opposite-type matches.
4. Polish	Seed data, loading/error states, responsive UI	The demo path works repeatedly without manual fixes.
5. Stretch	Images, maps, notifications, auth	Only if the core demo is already stable.

Demo script

1	Problem “Lost-and-found systems depend on people using exactly the same words and checking the right place.”
2	Lost report Create: “Black AirPods Pro charging case lost near the library.”
3	Found report Create: “Dark wireless earbud case found outside the library entrance.”
4	Reveal Open the lost item and show the found report as a high-confidence potential match.
5	MongoDB Show the stored documents and explain Vector Search + the additional ranking signals.

Definition of success

A judge can understand the problem, create or inspect a report, see a meaningful match, and understand exactly why MongoDB is essential - all within three minutes.